

# **Modern CLI Tools for Sysadmins**

**A Curated Collection of Tools to Transform Your Workflow**

**From the dev-env Project**

# What We'll Cover Today

- **Foundation:** Zsh & Zellij - Modern shell and terminal management
- **Navigation:** File finding, fuzzy search, and smart directory jumping
- **Search & Text:** Fast search and beautiful file viewing
- **Monitoring:** Process, system, network, and I/O monitoring
- **Version Control:** Terminal UIs for git
- **Containers:** Podman, distrobox, and container management
- **Development:** Editors, database clients, API tools
- **Utilities:** Helpers that make life easier

**All installed via one Ansible playbook - reproducible and version-controlled**

# Part 1: Foundation

## Z Shell & Oh-My-Zsh

# Z Shell (Zsh) + Oh-My-Zsh

## What is it?

Modern, feature-rich shell extending bash with powerful auto-completion, better globbing, and extensive customization. Oh-My-Zsh adds 200+ plugins and easy configuration.

## Why adopt it?

-  Command correction - typo detection and suggestions
-  Advanced tab completion - context-aware for commands, flags, paths
-  Shared history across all terminal sessions
-  Plugin ecosystem - git status, AWS, kubectl, and more

# Zsh: Key Features to Demo

## 1. Intelligent Auto-completion

```
kill -<TAB>          # Shows all signal options  
cd /u/l/b<TAB>       # Expands to /usr/local/bin
```

## 2. Command History Search

```
Ctrl+R              # Reverse history search with fzf  
# Type 'ssh' then ↑  # Shows only ssh commands
```

## 3. Plugins Included

- `zsh-autosuggestions` - Ghost text from history
- `zsh-syntax-highlighting` - Real-time validation (red/green)
- `zsh-fzf-history-search` - Fuzzy search with Ctrl+R

## Zsh: Spaceship Prompt

Shows critical info at a glance:

```
~/project main ↑1 !2 [+3 ~1 -1] # Git branch, unpushed commits, changes  
took 2s                          # Execution time for long commands
```

- Git branch and status automatically displayed
- Shows Python/Node versions when in project directories
- Execution time for commands > 1 second
- Customizable and beautiful

# Zellij

## Modern Terminal Multiplexer

# Zellij - Terminal Workspace Manager

## What is it?

Modern terminal multiplexer (like tmux) with better UX, tabs, panes, layouts, and session persistence.

## Why adopt it?

-  Session persistence - survive disconnects, perfect for SSH
-  No memorizing keybindings - on-screen hints
-  Built-in layouts - quickly split for monitoring
-  Better than screen/tmux - modern defaults, easier config



# Zellij: Essential Commands

## Session Management

```
zellij          # Start new session  
zellij attach   # Reconnect to session
```

## Pane Management (Ctrl+P)

- Split horizontally and vertically
- Resize with shortcuts shown on screen
- Close panes easily

## Tab Management (Ctrl+T)

- Create tabs for different tasks
- Rename tabs for organization

## Zellij: Real-World Use Case

### Demo Layout:

|                                                     |                                               |
|-----------------------------------------------------|-----------------------------------------------|
| Tab 1: Logs<br><code>tail -f /var/log/syslog</code> |                                               |
| Tab 2: Editor<br><code>vim nginx.conf</code>        | Tab 3: Commands<br><code>systemctl ...</code> |

**Disconnect → SSH back in → Still there!**

## Part 2: File & Directory Navigation

# FZF - Fuzzy Finder

## What is it?

Interactive command-line fuzzy finder. Filters files, history, processes in real-time.

## Why adopt it?

-  Find files without exact names
-  Search command history interactively
-  Integrates with git, vim, cd

## FZF: Key Bindings

```
Ctrl+R      # Search command history (fuzzy)
Ctrl+T      # Find file, insert path into command
Alt+C       # Fuzzy find directory and cd

# Pipe anything to fzf
ls /var/log | fzf
docker ps -a | fzf
```

### Example:

Type "ngnx" → matches "nginx", "nginx.conf", etc.

# Zoxide - Smart CD

## What is it?

Smarter cd that learns your most-visited directories. Jump with minimal typing.

## Why adopt it?

-  No more `cd ../../var/log/nginx` repeatedly
-  Jump to frequent dirs: `z ngi` → `/var/log/nginx`
-  Learns your habits, ranks by frequency

## Commands:

```
z nginx          # Jump to most frequent nginx directory
zi              # Interactive selection with fzf
zoxide query     # See ranked directories
```

# Eza & LSD - Modern LS

## What are they?

Modern `ls` replacements with colors, icons, git integration, and tree views.

## Why adopt them?

-  Git integration - see modified/staged files
-  Color-coded permissions
-  Tree view built-in
-  Icons for file types

```
eza -la          # Detailed listing with icons
eza --git        # Show git status
eza --tree --level=2  # Directory tree
```

# FD - Fast Find Alternative

## What is it?

User-friendly `find` alternative. Fast, simple syntax, respects `.gitignore`.

## Why adopt it?

-  10x faster than find
-  Simple syntax: `fd pattern`
-  Ignores `.git`, `node_modules` automatically
-  Colorized output

## Examples:

```
fd nginx          # vs find . -name "*nginx*"
fd -e conf        # Find all .conf files
fd -t d lib       # Find directories named 'lib'
fd -H .env        # Include hidden files
```



# Yazi - Terminal File Manager

## What is it?

Fast terminal file manager with vim keybindings, preview, and bulk operations.

## Why adopt it?

-  Visual file browsing
-  Preview text, images, PDFs
-  Bulk operations (copy/move/delete)
-  Keyboard-driven, no mouse needed

## Demo:

- Launch with `yazi`
- Navigate with hjkl or arrows
- Preview files in panel

# Broot - Tree View & Navigator

## What is it?

Interactive directory navigator with tree views, file sizes, fuzzy search for large directories.

## Why adopt it?

-  Handles huge directories without hanging
-  Combined tree + du functionality
-  Fuzzy search to filter tree
-  Quick navigation and cd

## Commands:

```
br                                # Launch broot
# Type to filter tree in real-time
:cd                               # Jump to selected directory
```

# Part 3: Search & Text Processing

# Ripgrep (rg) - Ultra-Fast Search

## What is it?

Blazing fast recursive search. Respects `.gitignore`. Replaces grep/ag/ack.

## Why adopt it?

-  10-100x faster than grep on large codebases
-  Skips .git, binaries, respects .gitignore automatically
-  Colored output with line numbers
-  Simple syntax, better defaults

## Examples:

```
rg "error" /var/log          # Search logs for "error"
rg -i "failed"              # Case-insensitive
rg -t py "import"          # Search only Python files
rg -A 3 -B 3 "error"        # Show 3 lines context
```

# Bat - Cat with Superpowers

## What is it?

`cat` clone with syntax highlighting, git integration, line numbers, and paging.

## Why adopt it?

-  Syntax highlighting for code/configs
-  Git integration - shows changes
-  Line numbers for reference
-  Automatic paging (no more `| less` )

## Examples:

```
bat /etc/nginx/nginx.conf          # Syntax highlighted
bat --style=numbers,changes script.py # Show git changes
bat --paging=never file.txt         # Quick view, no pager
```

# Hexyl - Hex Viewer

## What is it?

Beautiful hex viewer with colored output distinguishing ASCII, null bytes, binary data.

## Why adopt it?

-  Inspect binary files easily
-  Colored output for data types
-  Side-by-side hex + ASCII view

```
hexyl /bin/ls | head -50      # View binary
hexyl suspicious-file.bin    # Examine unknown files
```

# UP - Ultimate Plumber

## What is it?

Interactive pipe-building tool. Write complex commands interactively, see results in real-time.

## Why adopt it?

-  See pipe results as you type
-  Debug pipeline failures
-  Build complex grep/awk/sed chains iteratively

## Demo:

```
up /var/log/syslog
# Interactively build: grep error | grep -v INFO | awk '{print $1, $5}'
# Copy final command when done
```

# Part 4: Process & System Monitoring



# Procs - Modern Process Viewer

## What is it?

Modern `ps` replacement with colors, tree view, better default columns.

## Why adopt it?

-  Human-readable output with colors
-  Tree view of parent-child processes
-  Sensible defaults without complex flags
-  Built-in search/filter

## Examples:

```
procs                # All processes, colorized
procs nginx          # Search for nginx
procs --tree         # Process tree view
```

# Bottom - System Monitor

## What is it?

Cross-platform graphical system monitor for terminal. htop + iotop + nethogs combined.

## Why adopt it?

-  All-in-one: CPU, memory, disk, network, processes
-  Graphs and charts in terminal
-  Customizable views and layouts
-  Works over SSH

## Launch:

```
btm          # Full dashboard with graphs
```

# Nmon - Performance Monitor

## What is it?

System performance monitoring tool showing CPU, memory, disk I/O, network in one dashboard.

## Why adopt it?

-  Comprehensive view of all stats
-  Interactive - toggle views with keys
-  Recording mode for later analysis
-  Perfect for troubleshooting incidents

## Usage:

```
nmon                                     # Interactive mode
# Press: c (CPU), m (memory), d (disk), n (network)
```

# Iotop - I/O Monitoring

## What is it?

Shows I/O usage by processes. Like top, but for disk operations.

## Why adopt it?

-  Find I/O hogs instantly
-  Real-time disk reads/writes
-  Sort by I/O activity

## Usage:

```
sudo iotop                # Real-time disk I/O
# Sort by write (w), read (r)
```

**Use case:** Database slow? Check if logs are hammering the disk.

# Iftop - Network Bandwidth Monitor

## What is it?

Shows network bandwidth usage by connection. See which IPs/hosts consume bandwidth.

## Why adopt it?

-  Find bandwidth hogs instantly
-  Track active connections and speeds
-  Per-host statistics

## Usage:

```
sudo iftop -i eth0      # Monitor interface
# Press 't' to toggle views
```

# Gping - Graphical Ping

## What is it?

Ping with a graph. Visualizes ping times over time.

## Why adopt it?

-  Visual debugging of network latency
-  Ping multiple hosts simultaneously
-  Spot intermittent issues easily

## Examples:

```
gping 8.8.8.8 # Single host graph
gping 8.8.8.8 1.1.1.1 google.com # Compare multiple
```

Latency spikes appear as graph peaks - impossible to miss!

# Bandwhich - Bandwidth by Process

## What is it?

Shows which processes use network bandwidth, with connection details.

## Why adopt it?

-  Process-level network visibility
-  Real-time monitoring
-  See remote IPs, ports, protocols

## Usage:

```
sudo bandwhich          # Launch bandwidth monitor
```

**Use case:** Mystery bandwidth usage? Find the exact process responsible.

# Part 5: Version Control Tools



# GitUI

## What is it?

Blazing fast terminal UI for git (Rust). Keyboard-driven for commits, staging, branching, history.

## Why adopt it?

-  Speed - faster than lazygit, handles large repos
-  Keyboard-centric - no mouse, vim-like
-  Visual workflow - see changes, stage hunks
-  Async operations - background fetch

## Demo:

```
gitui
```

```
# Launch in git repo
```

# Lazygit

## What is it?

Simple terminal UI for git. More beginner-friendly than gitui.

## Why adopt it?

-  Clear labels and help text
-  Visual branch structure
-  Interactive rebase - squash, reword, reorder
-  Conflict resolution UI

## Demo:

```
lazygit
```

```
# Launch in git repo
```

# Part 6: Containers & Virtualization

# Podman

## What is it?

Daemonless container engine compatible with Docker. Rootless by default, more secure.

## Why adopt it?

-  No daemon - no single point of failure
-  Rootless containers - better security
-  Docker compatible - same commands, images
-  Systemd integration - generate units from containers

## Examples:

```
podman run -it ubuntu bash      # Run container
podman ps                      # List containers
podman generate systemd --name foo # Create systemd unit
```

# Podman-TUI

## What is it?

Terminal UI for Podman. Manage containers, images, volumes, networks visually.

## Why adopt it?

-  See all resources at a glance
-  Bulk operations on containers
-  View logs and stats inline
-  Faster than typing CLI commands

## Launch:

```
podman-tui          # Interactive container management
```

# Distrobox

## What is it?

Run any Linux distribution inside containers with host integration. Home, users, devices shared.

## Why adopt it?

-  Test on different distros (Fedora on Ubuntu, etc.)
-  Use packages from any distro's repos
-  Isolated dev environments per project
-  Seamless - apps run as if native

## Examples:

```
distrobox create --name fedora --image fedora:latest
distrobox enter fedora # Your home dir is shared!
sudo dnf install httpd # Install Fedora packages
```

# Part 7: Development & Productivity

# Helix - Modal Text Editor

## What is it?

Post-modern modal editor with built-in LSP, multiple cursors, tree-sitter parsing.

## Why adopt it?

-  Zero configuration - LSP works out of box
-  Multiple cursors - edit multiple locations
-  Built-in LSP - completions, diagnostics, go-to-def
-  Modern architecture - accurate syntax

## Usage:

```
hx /etc/nginx/nginx.conf          # Open file  
# Multiple cursors: C (add cursor below)  
# LSP features work automatically
```



# Lazysql - Database TUI

## What is it?

Terminal UI for SQL databases (MySQL, PostgreSQL, SQLite). Browse tables, run queries visually.

## Why adopt it?

-  Visual query building
-  Works over SSH, no GUI needed
-  Manage multiple databases
-  Safe - confirms destructive queries

## Launch:

```
lazysql
```

```
# Connect to database
```

# Slumber - API Client

## What is it?

Terminal-based HTTP/REST API client. Postman/Insomnia for the terminal.

## Why adopt it?

-  Test APIs without GUI, works over SSH
-  Save request collections
-  Environment variables (dev/staging/prod)
-  Response syntax highlighting

## Demo:

```
slumber                # Launch client
```

# NVM - Node Version Manager

## What is it?

Manage multiple Node.js versions on same system. Switch per project.

## Why adopt it?

-  Test on different Node versions
-  Per-project versions (.nvmrc)
-  Easy switching: `nvm use 18`
-  No sudo for global packages

## Commands:

```
nvm list          # Installed versions
nvm install 20    # Install Node 20
nvm use 18        # Switch to Node 18
node --version    # Verify
```

# Part 8: Utilities & Helpers

# TLDR - Simplified Man Pages

## What is it?

Community-driven man pages with practical examples. Skip to examples immediately.

## Why adopt it?

-  Quick reference with examples
-  Learn by example, not specs
-  1000+ commands covered
-  Faster than googling

## Examples:

```
tldr tar                # vs man tar (pages of text)
tldr rsync              # Common rsync use cases
tldr curl              # HTTP request examples
```

# Navi - Interactive Cheatsheets

## What is it?

Interactive cheatsheet tool. Search commands, fill placeholders interactively.

## Why adopt it?

-  Command templates with placeholders
-  Interactive variable prompts
-  Community cheatsheets available
-  Create team-specific references

## Demo:

```
navi                # Launch cheatsheet browser
# Search "ssh tunnel"
# Fill in variables interactively
# Execute or copy command
```

# Dust - Disk Usage Analyzer

## What is it?

Modern `du` that visualizes disk usage in a tree. Find space hogs instantly.

## Why adopt it?

-  Visual tree of directory sizes
-  Sorted by size (largest first)
-  Faster than `du` (parallel scanning)
-  Color-coded for visibility

## Examples:

```
dust /var/log          # Analyze directory
dust -d 3              # Limit depth for quick overview
```

# XH - HTTP Client

## What is it?

Friendly HTTP client with expressive syntax. Like HTTPie but faster (Rust).

## Why adopt it?

-  Human-friendly syntax (simpler than curl)
-  Automatic JSON serialization
-  Colored output
-  Download progress bars

## Examples:

```
xh httpbin.org/get # Simple GET
xh POST httpbin.org/post name=admin # POST with JSON
xh -d httpbin.org/image/png > image.png # Download file
```



# Espanso - Text Expander

## What is it?

Cross-platform text expander. Type shortcuts that expand to longer text.

## Why adopt it?

-  Time saver for common commands/emails
-  Ensure consistent templates
-  Works everywhere (terminal, browser, apps)
-  Dynamic expansions (date, command output)

## Examples:

```
:email      → Full email signature  
:date       → 2026-01-22  
:sshprod    → ssh -i ~/.ssh/prod.pem user@prod-server.com
```

# Watchexec - File Watcher

## What is it?

Executes commands when files change. Monitor configs, auto-restart services.

## Why adopt it?

-  Auto-restart services on config changes
-  Run tests automatically on save
-  Trigger alerts on log patterns
-  Simple syntax vs inotify-tools

## Examples:

```
watchexec -w /etc/nginx/nginx.conf -- nginx -t      # Test on change
watchexec -e py -- pytest                             # Run tests on save
```

# Asciinema - Terminal Recorder

## What is it?

Record and share terminal sessions. Perfect for documentation and demos.

## Why adopt it?

-  Lightweight - text-based, tiny files
-  Shareable - upload or self-host
-  Copy-paste enabled for viewers
-  Better than video (faster, searchable, smaller)

## Usage:

```
asciinema rec demo.cast          # Start recording
# Run commands...
exit                             # Stop recording
asciinema play demo.cast         # Replay
```

## More Utilities

**SSH-List** - SSH connection manager with fuzzy search

```
ssh-list          # Fuzzy search SSH config
```

**CCZE** - Log colorizer

```
tail -f /var/log/syslog | ccze -A    # Colorized logs
```

Makes ERROR/WARNING stand out in red/yellow!

# Getting Started

## Installation & Adoption Strategy

# Installation

## One Command Setup:

```
./setup.sh && source activate.sh && \  
ansible-playbook dev-setup.yml --ask-become-pass
```

## Selective Installation (use tags):

```
ansible-playbook dev-setup.yml --tags "zsh,fzf,ripgrep"
```

All versions pinned in `vars/versions.yml`

# Recommended Adoption Path

## Week 1: Foundation

- Install Zsh + Oh-My-Zsh
- Start using Zellij for session management
- Adopt fzf for command history (Ctrl+R)

## Week 2: Navigation & Search

- Replace ls → eza/lsd
- Use ripgrep instead of grep
- Try bat instead of cat
- Start using zoxide to jump directories

# Adoption Path (continued)

## Week 3: Monitoring

- Use procs instead of ps
- Install bottom for troubleshooting
- Try gping for network diagnostics

## Week 4+: Specialized Tools

- Adopt gitui/lazygit for version control
- Try helix for quick edits
- Explore container tools (podman, distrobox)
- Add utilities based on your workflow



## Creating Aliases for Transition

Add to `.zshrc` :

```
alias ls='eza'  
alias cat='bat'  
alias find='fd'  
alias grep='rg'  
alias ps='procs'  
alias du='dust'
```

Gradual replacement of muscle memory!

## Team Adoption Tips

1. **Start non-destructive** - Try tools that don't replace critical commands first
2. **Share configurations** - Commit `.zshrc` and configs to team repo
3. **Create cheatsheets** - Use Navi for team-specific commands
4. **Demo in standups** - Show one tool per week
5. **Pair sessions** - Help teammates get comfortable

## Version Management:

```
ansible-playbook version-checks.yml      # Check for updates
```

# Conclusion

## Key Takeaways

- ✅ **Productivity boost** - These tools genuinely make tasks faster and easier
- ✅ **Better UX** - Modern tools prioritize usability and helpful defaults
- ✅ **Minimal risk** - Most are drop-in replacements, easy to revert
- ✅ **Reproducible setup** - Ansible playbook = consistent environments

## Next Steps

1. **Clone the repo** and run setup
2. **Start with foundation:** Zsh + Zellij + FZF
3. **Gradually adopt** tools matching your workflow
4. **Share feedback** and contribute improvements

# Questions?

## Let's Transform How We Work

**Repository:** <https://github.com/jquacinella/dev-env>

**Documentation:** Check README.md for detailed usage

## Final Thought

"These tools represent years of open-source development focused on solving real sysadmin problems.

Don't try to adopt everything at once—pick 3-5 tools that address your pain points and go from there.

The time investment pays back quickly."

**Thank you!**